

Project Title

by

Author(s) Name

Stevens.edu

October 11, 2024

© Author(s) Name
Stevens.edu
ALL RIGHTS RESERVED

Project Title

Author(s) Name
Stevens.edu

This document provides the requirements and design details of the PROJECT. The following table (Table 1) should be updated by authors whenever major changes are made to the architecture design or new components are added. Add updates to the top of the table. Most recent changes to the document should be seen first and the oldest last.

Table 1: Document Update History

Date	Updates
08/22/2023	DDM: Updated dsnManual.tex with <i>newcommand(s){}</i> for easier references of requirements, figures, and other labels.
10/25/2023	DDM: Added chapters on use cases (Chapter ??) and user stories (Chapter ??).
10/11/2023	DDM: Added chapters on requirements (Chapter ??) and glossary.
09/18/2023	DDM: Added chapter on development plan (Chapter ??).

Table of Contents

1	Team	
	– <i>Enoch Chan</i>	1
2	gitHomework	
	– <i>Enoch</i>	4
3	UML Class Modeling	
	– <i>Enoch</i>	6
3.1	2.1 Exercise	6
3.2	2.2 Exercise	6
3.3	2.3 Exercise	6
3.4	2.4 Exercise	8
3.5	ProductSale	9
4	Software Execution Model	
	– <i>Enoch</i>	12
4.1	5.1 Exercise	12
4.2	5.2 Exercise	13
4.3	5.3 Exercise	15
	Bibliography	18

List of Tables

1	Document Update History	iii
4.1	Service Demands for Key Functions	17

List of Figures

1.1	Diagram 1	2
1.2	Diagram 2	3
2.1	git Homework Screenshot	5
3.1	Diagram 1	6
3.2	Diagram 1	6
3.3	Results Screenshot	10
3.4	UML	11
4.1	Diagram 1	15
4.2	Diagram 2	16

Chapter 1

Team

– Enoch Chan

My name is Enoch. I am a software engineer major at Stevens Institute of Technology. I was born and raised in Staten Island, New York. I like to play volleyball and go to the gym. I am currently working at the Front Desk of Fitness Factory on 2nd Street. Below are my .EPS figures:

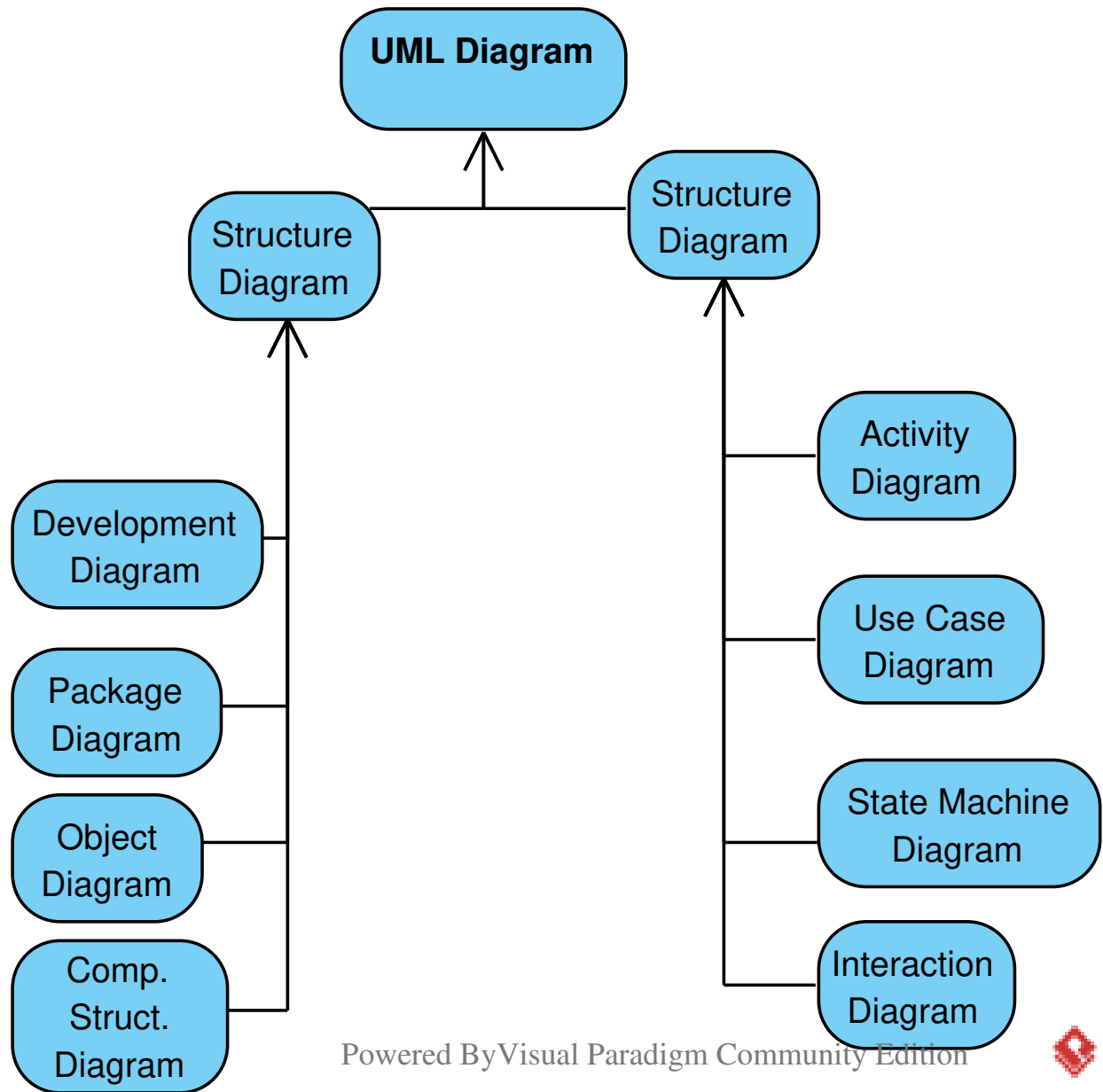


Figure 1.1: Diagram 1



Figure 1.2: Diagram 2

Chapter 2

gitHomework

– Enoch

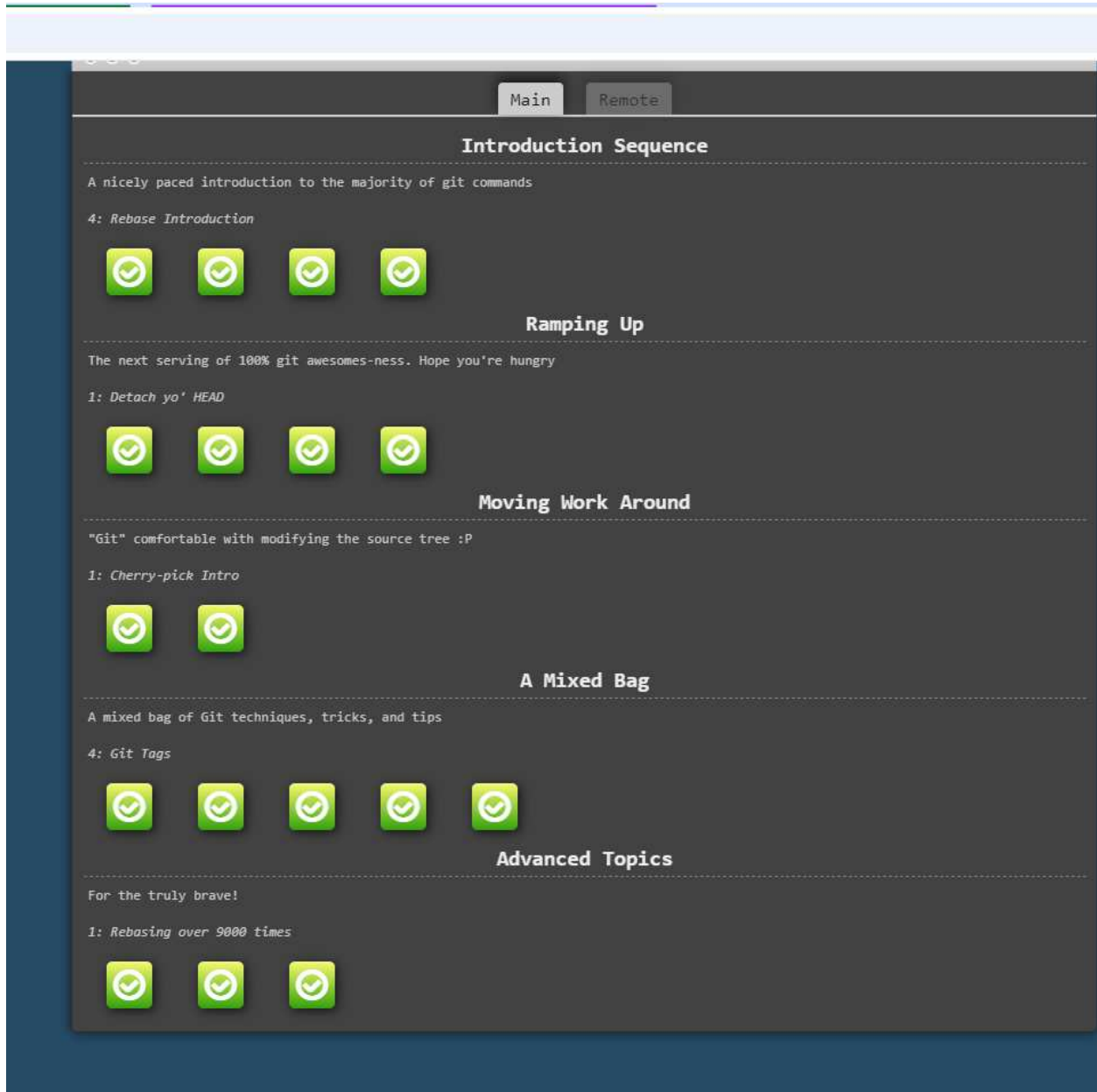


Figure 2.1: git Homework Screenshot

Chapter 3

UML Class Modeling

– Enoch

3.1 2.1 Exercise



Figure 3.1: Diagram 1

3.2 2.2 Exercise

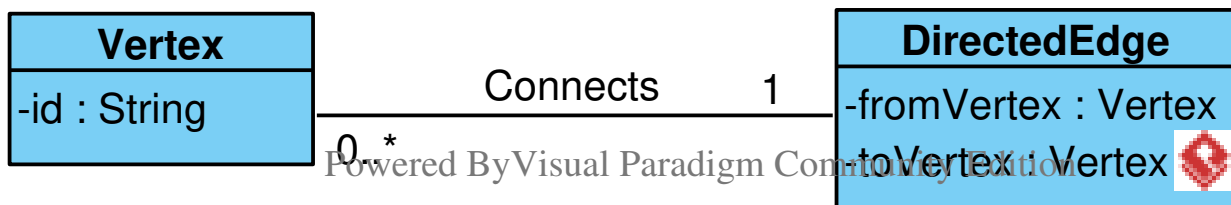


Figure 3.2: Diagram 1

3.3 2.3 Exercise

Window → **ScrollingWindow**

Association: A **ScrollingWindow** is a special kind of **Window**.

Description: The ScrollingWindow allows for scrolling within the Window area.

Canvas → Shape

Association: A Canvas can have many shapes.

Description: The Canvas holds different shapes that can be drawn, like lines or closed shapes.

Canvas → ScrollingCanvas

Association: A ScrollingCanvas is a type of Canvas.

Description: The ScrollingCanvas can scroll to show more of the drawing area.

Canvas → Line

Association: A Canvas can have many lines.

Description: A Line is drawn on the Canvas and is defined by its start and end points.

Shape → ClosedShape

Association: A ClosedShape is a type of Shape.

Description: A ClosedShape is a shape that forms a complete boundary, like a circle or polygon.

Shape → Line

Association: A Line is a type of Shape.

Description: A Line is a straight shape with a start and end point.

ClosedShape → Polygon

Association: A ClosedShape can be a Polygon.

Description: A Polygon is a shape with multiple sides.

ClosedShape → Ellipse

Association: A ClosedShape can be an Ellipse.

Description: An Ellipse is an oval shape.

Canvas → Panel

Association: A Canvas can be part of a Panel.

Description: The Panel can hold a Canvas and other interface elements.

Panel → PanelItem

Association: A Panel has many PanelItems.

Description: PanelItems are objects within the Panel, such as buttons or text fields.

Panel → Button

Association: A Panel can have many Buttons.

Description: A Button is a clickable item in the Panel.

Panel → TextItem

Association: A Panel can have TextItems.

Description: A TextItem displays text or allows the user to input text.

Panel → ChoiceItem

Association: A Panel can have ChoiceItems.

Description: A ChoiceItem allows the user to choose from a list of options.

ChoiceItem → ChoiceEntry

Association: A ChoiceItem has multiple ChoiceEntries.

Description: A ChoiceEntry is one of the options in a ChoiceItem.

PanelItem → Event

Association: A PanelItem can trigger an Event.

Description: When a PanelItem (like a button) is clicked, it triggers an Event.

ScrollingWindow → Scroll

Association: A ScrollingWindow allows scrolling.

Description: The scroll function moves the visible area of the ScrollingWindow.

TextWindow → Text

Association: A TextWindow manages text.

Description: The TextWindow allows text to be inserted or deleted.

3.4 2.4 Exercise

Customer → MailingAddress

Association: A Customer has one or more MailingAddresses.

Description: The MailingAddress is associated with the Customer and holds information like address and phone number for communication purposes.

MailingAddress → CreditCardAccount

Association: A CreditCardAccount can have one or more MailingAddresses.

Description: The MailingAddress is linked to the CreditCardAccount for billing and correspondence.

CreditCardAccount → Statement

Association: A CreditCardAccount can generate multiple Statements.

Description: The Statement shows important details like the payment due date, finance charges, and minimum payment, summarizing the account's activity.

CreditCardAccount → Institution

Association: A CreditCardAccount is associated with one Institution.

Description: The Institution is the financial entity responsible for the CreditCardAccount, holding details like name, address, and phone number.

CreditCardAccount → Transaction

Association: A CreditCardAccount can have multiple Transactions.

Description: Each Transaction represents an action on the CreditCardAccount, such as purchases, cash advances, fees, or adjustments.

Statement → Transaction

Association: A Statement includes multiple Transactions.

Description: Each Transaction is part of a Statement, showing detailed records of charges, payments, or fees on the account.

Transaction → Purchase

Association: A Transaction can be a Purchase.

Description: A Purchase is a type of Transaction where goods or services are bought from a Merchant.

Transaction → Fee

Association: A Transaction can be a Fee.

Description: A Fee is a type of Transaction that includes extra charges like late fees or service fees.

Transaction → Interest

Association: A Transaction can include Interest.

Description: Interest is a finance charge added as a result of carrying a balance on the CreditCardAccount.

Transaction → CashAdvance

Association: A Transaction can be a CashAdvance.

Description: A CashAdvance is a type of Transaction where cash is withdrawn from the credit card account.

Transaction → Adjustment

Association: A Transaction can be an Adjustment.

Description: An Adjustment modifies or corrects a previous transaction, either as a credit or debit to the account.

Purchase → Merchant

Association: A Purchase is linked to a Merchant.

Description: The Merchant is the business or entity from which a Purchase is made, identified by name.

3.5 ProductSale

```
1 from __future__ import annotations
2 from typing import List
3
4 # Forward reference for class Sale
5 class Product:
6     __lastSale: Sale = None
7     __inventory: int = 0 # Attribute to track inventory
8
9     def __init__(self, inventory: int = 0):
10         self.__inventory = inventory
11
12     def setLastSale(self, lastSale: Sale):
13         self.__lastSale = lastSale
14
15     @property
16     def getLastSale(self) -> Sale:
17         return self.__lastSale
18
19     @property
20     def getInventory(self) -> int:
21         return self.__inventory
22
```

```
23     def decreaseInventory(self, amount: int):
24         if self.__inventory >= amount:
25             self.__inventory -= amount
26         else:
27             raise ValueError(f"Not enough inventory to sell {amount} units.
                Current inventory: {self.__inventory}")
28
29 # No forward reference needed since Product is already defined
30 class Sale:
31     __saleCounter = 0 # Static variable to track number of sales
32     __productSold: List[Product] = None
33
34     def __init__(self, products: List[Product], quantities: List[int]):
35         Sale.__saleCounter += 1
36         self.__saleNumber = Sale.__saleCounter
37         self.__productSold = products
38
39         for index, product in enumerate(products):
40             product.decreaseInventory(quantities[index])
41             product.setLastSale(self)
42
43     @property
44     def getSaleNumber(self) -> int:
45         return self.__saleNumber
46
47 # Example usage
48
49 productOne = Product(inventory=100)
50 productTwo = Product(inventory=50)
51
52 # Sale 1: Selling 5 units of productOne and 3 units of productTwo
53 saleOne = Sale([productOne, productTwo], [5, 3])
54
55 # Sale 2: Selling 2 units of productOne
56 saleTwo = Sale([productOne], [2])
57
58 print(f"ProductOne Inventory: {productOne.getInventory}, Last Sale Number: {
    productOne.getLastSale.getSaleNumber}")
59 print(f"ProductTwo Inventory: {productTwo.getInventory}, Last Sale Number: {
    productTwo.getLastSale.getSaleNumber}")
```

```
'c:\Users\enoch\.vscode\extensions\ms-python.debugpy-2024.10.0-win32-x64\bundle\libs\debugpy\adapter\..\..\debugpy\launcher' '59641' '--' 'c:\Users\enoch\Down
loads\PythonClasses\ProductSale.py'
ProductOne Inventory: 93, Last Sale Number: 2
ProductTwo Inventory: 47, Last Sale Number: 1
```

Figure 3.3: Results Screenshot

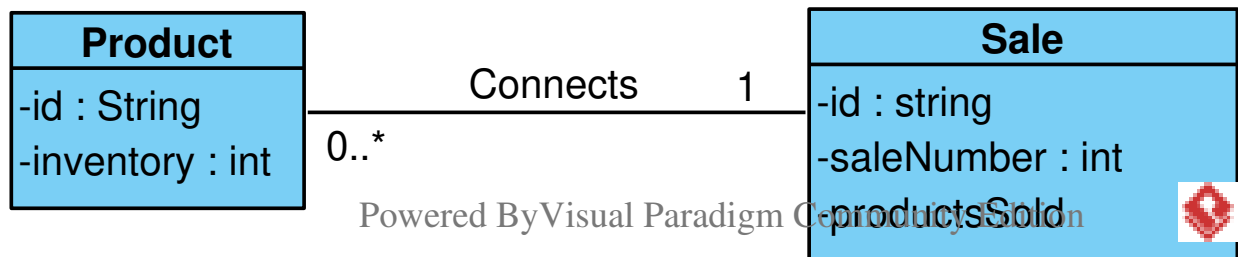


Figure 3.4: UML

Chapter 4

Software Execution Model

– Enoch

4.1 5.1 Exercise

Best, Worst, and Average Demand Times

- **Best Demand Time:**

$$\text{Best Demand Time} = \text{BASE} + 4 \times (P_3 + \text{PATH 3} + P_0) = 19.354\text{s}$$

- **Worst Demand Time:**

$$\text{Worst Demand Time} = \text{BASE} + 4 \times (P_3 + \text{PATH 4} + P_0) = 98.862\text{s}$$

- **Average Demand Time:**

$$\text{PATH AVERAGE} = (0.1 \times P_1) + (0.2 \times P_2) + (0.3 \times P_3) + (0.4 \times P_4) = 10.8094\text{s}$$

$$\text{Average Demand Time} = \text{BASE} + 4 \times (P_3 + \text{PATH AVERAGE} + P_0) = 60.5676\text{s}$$

Total Demand Times for CPU, Disk, and Network

- **CPU Demand Time:**

$$\text{CPU} = (3 \times (400 + 1500 + 10) + 1 \times (100 + 200 + 200 + 20 + 35 + 220)) \times 0.0001 = 0.6505\text{s}$$

- **Disk Demand Time:**

$$\text{DISK} = (2 \times 150 + 1 \times (2 + 10 + 2)) \times 0.02 = 6.28\text{s}$$

- **Network Demand Time:**

$$\text{NETWORK} = (1 \times (2 + 1) + 1 \times (100 + 2 + 2 + 100 + 30 + 10)) \times 0.01 = 2.47\text{s}$$

4.2 5.2 Exercise

Problem Statement

In this problem, we calculate the demand times for **Work**, **DB**, and **MSG** based on the given per-unit demand values:

$$\text{Work} = 0.0075\text{s/unit}$$

$$\text{DB} = 0.58\text{s/unit}$$

$$\text{MSG} = 0.1575\text{s/unit}$$

We then determine the demand times of each component of the system using these per-unit values.

Component Demand Time Calculations

The demand times for each component are calculated as follows:

$$\text{Initialize} = 2 \times \text{Work} + \text{DB} + \text{MSG} = 0.7525\text{s}$$

$$\text{Switch} = 1 \times \text{Work} = 0.0075\text{s}$$

$$\text{UpdateMirrorNY} = 5 \times \text{Work} + 2 \times \text{DB} + 2 \times \text{MSG} = 1.5125\text{s}$$

$$\text{UpdateMirrorLA} = 4 \times \text{Work} + 2 \times \text{DB} + 2 \times \text{MSG} = 1.505\text{s}$$

$$\text{QueryMirrorNY} = 3 \times \text{Work} + \text{DB} + 2 \times \text{MSG} = 0.9175\text{s}$$

$$\text{QueryMirrorLA} = 4 \times \text{Work} + \text{DB} + 2 \times \text{MSG} = 0.925\text{s}$$

$$\text{WriteMirrorNY} = 3 \times \text{Work} + \text{DB} + 2 \times \text{MSG} = 0.9175\text{s}$$

$$\text{WriteMirrorLA} = 4 \times \text{Work} + \text{DB} + 2 \times \text{MSG} = 0.925\text{s}$$

Base Demand Time Calculation

The **Base Demand Time** is calculated as the sum of the demand times for Initialize and Switch:

$$\text{BASE} = \text{Initialize} + \text{Switch} = 0.76\text{s}$$

Specific Demand Time Calculations

The demand times for specific operations are:

- **Update:** This will be calculated in specific demand cases.
- **Query:**

$$\text{Query} = \text{QueryMirrorNY} = 0.9175\text{s} \quad (\text{Series Operation})$$

- **Write:**

$$\text{Write} = \text{WriteMirrorLA} = 0.925\text{s} \quad (\text{Parallel Operation})$$

Best, Worst, and Average Demand Times

- **Best Demand Time:** To find the best demand time, we follow the fastest time path:

$$\text{Best Demand Time} = 2 \times (\text{BASE} + \text{QUERY}) = 3.355\text{s}$$

- **Worst Demand Time:** To find the worst demand time, we follow the slowest time path using the Update path:

$$\text{Worst Demand Time} = 2 \times (\text{BASE} + \text{SWITCH} + \text{UpdateMirrorNY}) = 4.56\text{s}$$

- **Average Demand Time:** The average demand time is calculated as follows:

$$\text{AVERAGE UPDATE} = \text{SWITCH} + 0.7 \times \text{UpdateMirrorNY} + 0.3 \times \text{UpdateMirrorLA} = 1.51775\text{s}$$

$$\text{AVERAGE} = \text{BASE} + 0.5 \times \text{AVERAGE UPDATE} + 0.4 \times \text{QUERY} + 0.1 \times \text{WRITE} = 3.95675\text{s}$$

4.3 5.3 Exercise

1. Sequence Diagram

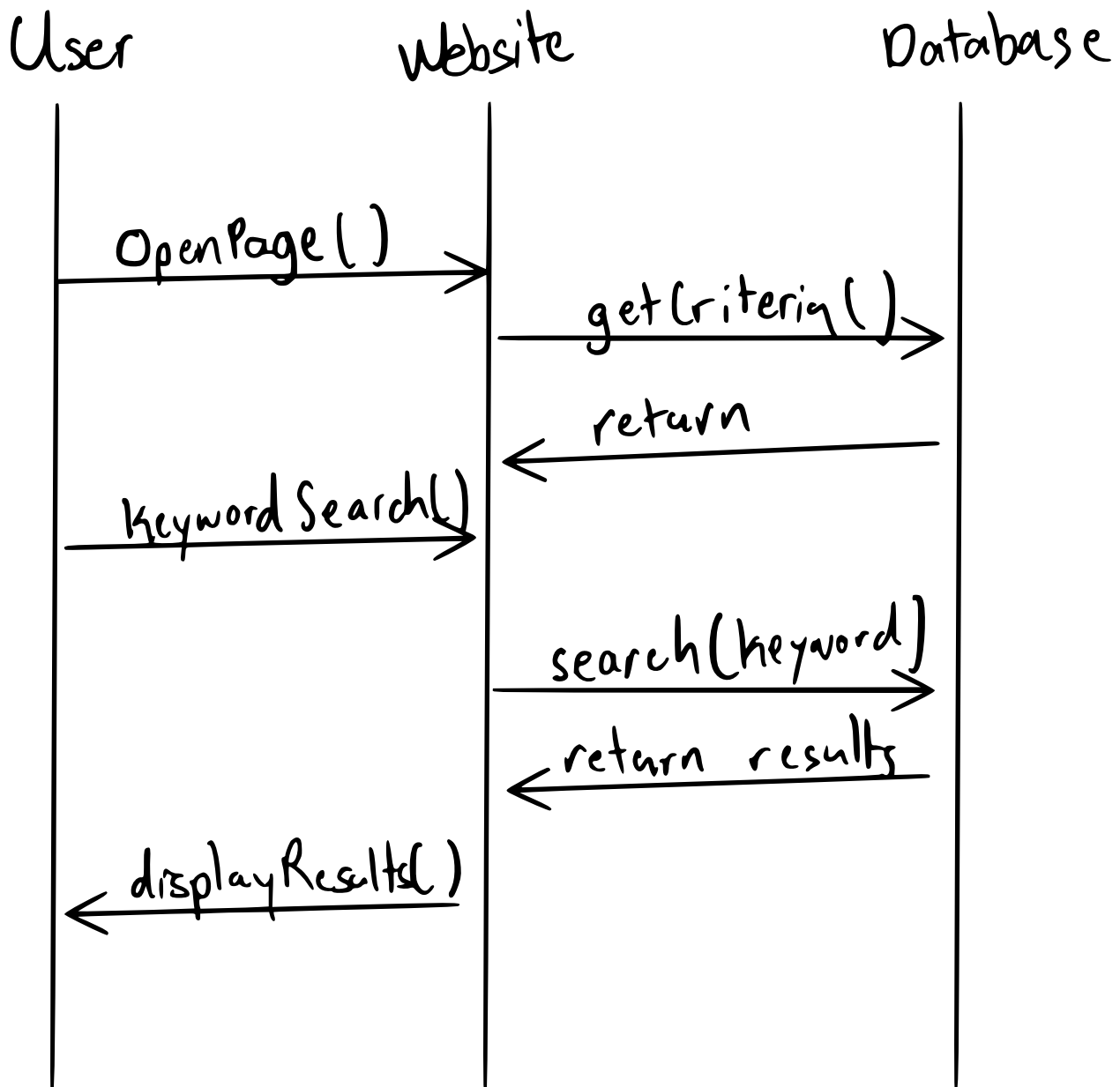


Figure 4.1: Diagram 1

The sequence diagram shows how the search is conducted between the three different objects: **User**, **Website**, and **Database**. The user can instantiate three types of searches:

1. **keywordSearch()** - Initiates a keyword-based search.

2. **isbnSearch()** - Initiates a search based on ISBN.

3. **authorNameSearch()** - Initiates a search based on the author's name.

The sequence involves the following steps:

- **User** opens the page by calling **OpenPage()** on **Website**.
- **Website** retrieves search criteria from **Database** by calling **getCriteria()**.
- **User** initiates a search type (e.g., **keywordSearch()**).
- **Website** requests data from **Database**.
- **Website** calls **displayResults()** to show results to **User**.

2. Search Execution Model

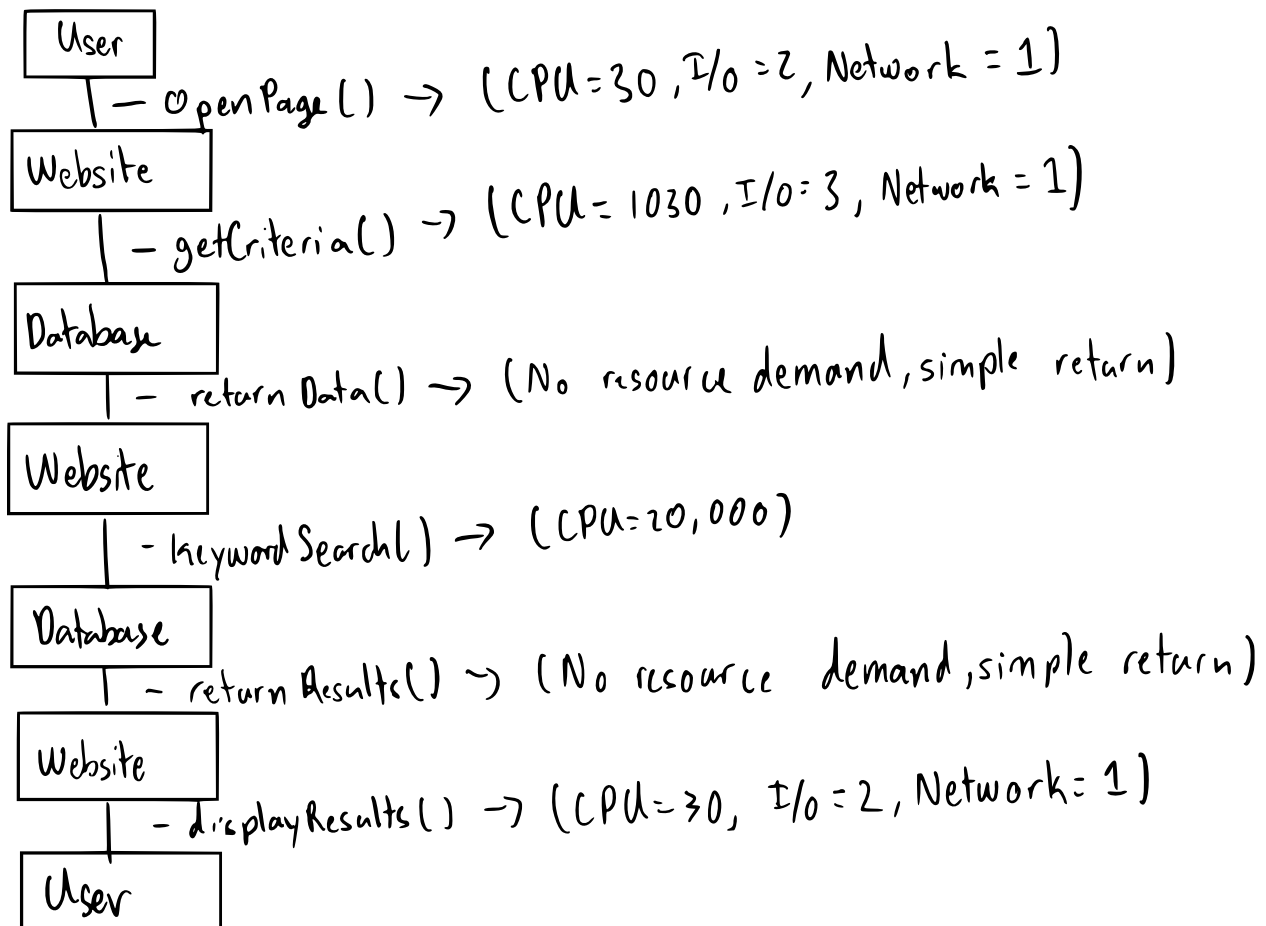


Figure 4.2: Diagram 2

The execution model is created based on the sequence diagram. The table below summarizes the service demands:

Processing Step	2x CPU	1x I/O	1x Network
OpenPage()	30	2	1
getCriteria()	1030	3	1
keywordSearch()	20000	0	0
isbnSearch()	0	0	0
authorNameSearch()	0	0	0
displayResults()	30	2	1

Table 4.1: Service Demands for Key Functions

3. Assign Probabilities

The following probabilities are assigned for each search type:

- Probability of **keywordSearch()** = 0.7
- Probability of **isbnSearch()** = 0.2
- Probability of **authorNameSearch()** = 0.1

The constraint is that the sum of probabilities must equal 1.

4. Calculation of Worst, Best, and Average Times

- **Worst-case time:** The **keywordSearch()** function has the highest service demand with 20,000 units for CPU. The time is calculated as:

$$T_{\text{keywordSearch}} = 20,000 \times 0.001 \text{ ms} = 20 \text{ ms}$$

- **Best-case time:** The **isbnSearch()** and **authorNameSearch()** functions have zero service demands, making them the fastest with a time of:

$$T_{\text{isbnSearch}} = 0 \text{ ms}$$

- **Average time:** Calculated as a weighted average based on the probabilities:

$$\text{Average time} = (0.6 \times 20 \text{ ms}) + (0.3 \times 0 \text{ ms}) + (0.1 \times 0 \text{ ms}) = 12 \text{ ms}$$

Bibliography

Index

Chapter

gitHomework, [4](#)

Software Execution Model, [12](#)

Team, [1](#)

UML Class Modeling, [6](#)

glossary, [4](#), [6](#), [12](#)

introduction, [1](#)